

Master's Thesis
*Markov-Switching models &
crypto-currencies*

Carl Paulus

Markow-Switching models & crypto-currencies

Master's Thesis

at Frankfurt School of Finance & Management

Supervised by

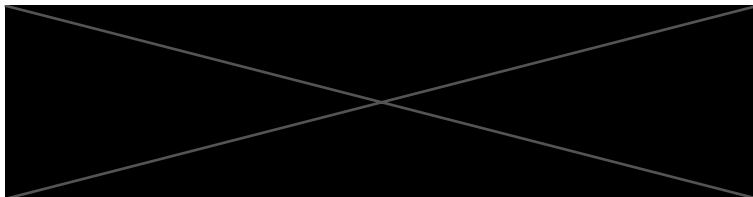
Prof. Dr. Jean-David Fermanian (Ensaie-Crest & Bitscor)

Prof. Dr. Lucas Böttcher (Frankfurt School of Finance & Management)

Submitted by

Carl Paulus

Master in Applied Data Science



Frankfurt am Main, July 2023

Contents

1	Acknowledgements	4
2	Introduction	5
3	Literature review	7
4	Data	10
4.1	Yahoo Finance	11
4.2	Stationarity tests	13
4.3	Introduction of regime changes	17
4.4	Skewness & kurtosis	22
4.5	Autocorrelations	23
5	MS-GARCH	25
5.1	The Model	25
5.2	Calculation of a likelihood function	28
5.3	Path dependencies in MS-GARCH models	31
6	Results	35
6.1	Parameters & conditional means	35
6.2	Regime persistence	36
6.3	Mean variances per regime	38
7	Conclusion	39
8	Bibliography	40
9	Appendix	41

1 Acknowledgements

Before starting, I would like to thank the highly valuable persons who have supported me during this long period of academic research. This thesis has been an important source of learning, enabling me to push back my limits on many occasions.

First and foremost, I would like to thank Prof. Dr. Jean David Fermanian for helping me in clarifying my topic of research and tackling the multiple mathematical challenges resulting from it. His invaluable advice, distilled strategically nourished my daily progress and helped me to get out of various tricky situations.

Prof. Dr Lucas Böttcher for following me throughout the development of the dissertation, with precise instructions on the academic rigour required, as much in terms of forms than in terms of content.

Mr Hugo Inzirillo, for guiding and inspiring me when I was reaching the end of my scope of capabilities. In fact, I was able to challenge my computer skills and this subject allowed me to surpass myself and make considerable progress.

My mentors from my previous internships at BNP Paribas (QIS - Quantitative Investment Strategy team in London), at UniCredit (EQD - Equity Derivatives Trading team in Munich), at Commerzbank (IT team of Group Treasury in Frankfurt), for enabling me to acquire the computer development and mathematical skills necessary for this subject.

Finally, I would like to thank all those who helped me push back my limits, because the subject was ambitious. Today, I am honoured to be able to present this Master's Thesis, which has been an immense source of learning in mathematics, statistics and computer science.

Abstract

Since the introduction of Markov-Switching models for the econometric analysis of economic cycles by Hamilton [9], this family of models has been extensively studied in the academic literature and among practitioners. Markov-switching models offer a powerful tool for capturing the real-world behaviour of time series data. The natural idea of extending such ideas to financial econometrics was not so straightforward, due to some well-known empirical features of asset returns: fat tails, skewness, and particularly volatility clustering. Since the latter feature is traditionally captured by GARCH-type models, some authors have proposed Markov-Switching GARCH (MS-GARCH, to be short) models to combine the idea of different regimes/cycles and the persistence of return volatilities. In this paper, we will apply MS GARCH on daily log returns' volatilities of the Bitcoin (BTC), assuming three underlying regimes. Our innovation comes from the fact that we will not follow the standard hypothesis that the conditional means are zero. We will use different conditional means across regimes.

2 Introduction

Today, it is common to use different time series models to analyse the dynamic behaviour of economic and financial variables. The most common choices are linear models. These models have been quite successful in many applications, except for complex economic datasets. The need for more complex models with multiple unconditional variables has arisen. Linear models naturally fail to represent many nonlinear dynamical patterns, such as asymmetries, amplitude dependencies, and volatility clustering. For example, GDP growth rates tend to fluctuate at higher levels and for longer periods of time during upswings, while staying at relatively lower levels and for shorter periods of time during contractions. With that kind of data, it is unrealistic to expect one linear model to capture these different behaviours.

The last two decades have seen a rapid increase in the development of nonlinear time series models. However, nonlinear time series models have their own limitations. The first problem we can think of is that nonlinear models are often difficult to implement. Secondly, most nonlinear models are designed to describe specific nonlinear data patterns and may therefore not be as flexible as one would wish. This shows that the dataset is strongly

correlated to the success of the model. A model would perfectly fit to some time series, but not others. However, artificial neural network models are an exception. They are able to catch any nonlinear pattern thanks to their "universal approximation" properties.

3 Literature review

The first researcher who introduced Markov switching within the world of econometrics was Hamilton. His first paper was released in 1989 and two complementary studies followed, Engel and Hamilton (1990), Hamilton and Susmel (1994). Initially, Hamilton focused on the mean behavior of variables. Since then, the Markov switching model and its variants have been applied to all sorts of economic dataset i.e. Goodwin (1993), Engel (1994), Diebold, Lee and Weinbach (1994), Kim and Nelson (1998), to name a few. More specifically, Taiwan’s business cycles have been a much studied: Huang (1999), Rau, Lin and Li (2001), Hsu and Kuan (2001) to name a few.

The Markov-switching model of conditional mean being highly relevant, researchers have naturally incorporated it into conditional variance models. This is how MS-GARCH was born. The Generalized Autoregressive Conditional Heteroskedasticity (GARCH) was first created by Engle (1982) and later Bollerslev (1986). Hamilton and Susmel (1994), Cai (1994) and Gray (1996) studied variants of ARCH/GARCH models with MS.

Within the cryptocurrency space, some researchers have found out that traditional models alone, including GARCH-types, can poorly estimate / forecasts volatility and returns. Bias is higher when regime changes in the volatility dynamic is ignored. This was specifically analyzed by Lamoureux and Lastrapes, 1990, and Bauwens et al., 2014. In fact, Bariviera (2017) finds that “Bitcoin returns exhibit some form of regime change, suggesting that regime-switching models could more adequately capture the volatility dynamics.”. Following this path of studies focusing on volatility dynamics of the Bitcoin returns, Chu et al. (2017) show evidence that volatility clustering combined to GARCH-type specifications “provides the best in-sample performance”.

Now, we will dig further into studies closer to the topic of this master’s thesis and detail researchs from Ardia, Bluteau, Rüede (2019), Haas, Mittnik, Paolletta (2004), Caporale, Zekokh (2019), Bollen, Gray, Whaley (2000) and Klaassen (2001).

Ardia, Bluteau and Rüeder’s work tests whether MS-GARCH models capture any regime changes in the Bitcoin volatility dynamics and outperform single-regime GARCH specifications in Value-at-Risk (VaR) forecasting. Extending the work of Katsiampa (2017), they consider various scedastic functions, error distributions, and specifications for up to three regimes, leading to a total of 18 models estimated using a Bayesian approach. Their results identify regime changes in the GARCH dynamics of Bitcoin. A two-regime specification for an asymmetric GARCH model with skewed and fat-tail conditional distribution leads to the best fit in-sample. The inverted leverage effect is observed in all volatility regimes. We also find that MSGARCH specifications outperform single-regime models for VaR forecasting.

Haas, Mittnik and Paolletta acknowledge that MS gives rise to a plausible interpretation of nonlinearities. However, they consider that GARCH-type models remain ubiquitous in order to allow for nonlinearities associated with time-varying volatility. Existing methods of combining the two approaches are unsatisfactory, as they either suffer from severe estimation difficulties or else their dynamic properties are not well understood. They present a new Markov-switching GARCH model that overcomes both of these problems. Concretely, they derive dynamic properties as well as their implications for the volatility process discussed. They argue that the disaggregation of the variance process offered by the new model is more plausible than in the existing variants. The approach is illustrated with several exchange rate return series. Their results suggest that a promising volatility model is an independent switching GARCH process with a possibly skewed conditional mixture density.

Coming to the study of Caporale and Zekokh, various findings are to mention. Firstly, the analysis shows that cryptocurrencies exhibit high volatility and leverage effects (at least in one regime). Secondly, the authors find that two-regime models yield better results in terms of prediction for both VaR and ES. Thirdly, mixture distribution models outperform Markov-switching ones. Fourthly, the analysis indicates that using single-regime GARCH models may yield incorrect VaR and ES predictions. Finally, the authors suggest that using model specifications allowing for asymmetries and regime switching can improve risk-management, portfolio optimization, and pricing of derivative securities for cryptocurrencies.

Bollen, Gray, Whaley finds that regime-switching models are better at describing the time series of exchange rates than competing models. The study also finds that market prices reflect some regime-switching information, but not fully. The authors also conducted a trading strategy experiment, which confirmed that the improved descriptive power of the regime-switching model may explain nothing more than perturbations within an option's bid-ask spread. Overall, the study provides insights into the dynamics of foreign exchange rates and the use of regime-switching models and exchange-traded currency options in forecasting and trading.

They also find that a path-dependency problem arises when regime probabilities are used in intermediate computations in the lattice. This is because regime probabilities are dependent on the particular series of observed changes in the underlying variable. To avoid this problem, the authors compute two conditional option values at each node, where the conditioning information is the prior regime. This approach allows the authors to value options in the standard way, iterating backward from the terminal array of nodes. For earlier nodes, conditional option values will depend on regime persistence since the persistence parameters are equivalent to future regime probabilities in a conditional setting.

Klaassen focuses on the prevalent usage of GARCH models among researchers for generating volatility forecasts. By analysing data pertaining to three USD exchange rates, it is demonstrated that these forecasts tend to be overly high during periods of volatility. The author argues that this phenomenon can be attributed to the enduring impact of shocks in GARCH forecasts. To address the issue of volatility persistence and introduce greater flexibility, the paper proposes a generalized version of the GARCH model that distinguishes between two regimes featuring distinct levels of volatility. Within each regime, GARCH effects are allowed to operate. This adaptation, known as the Markov regime-switching GARCH model, offers notable advancements over existing variations, particularly by facilitating a convenient recursive procedure for multi-period-ahead volatility forecasting. Through empirical analysis, it is shown that this model effectively resolves the issue associated with high single-regime GARCH forecasts, resulting in significantly improved out-of-sample volatility forecasts.

4 Data

For this Master’s thesis, we focused our research on four cryptocurrencies: Bitcoin, Ethereum, Ripple and Cardano. Preliminary statistical tests are mandatory in order to pursue a valuable and unbiased work.

However, we first need to know what time series characteristics would suit to the model in order for it to be performing. The model understates that multiple equations / structures can describe a time series behavior in distinct regimes. The model must be well parameterized in order to be able to find which observation corresponds to which regime. Switching between regimes, allows the model to capture complex patterns. The main feature of the MS model is that an unobservable state variable following a Markov chain controls the switching mechanism. The value at time t depends on $t-1$ as the Markovian property implies. As such, a regime may prevail at a certain period of time and with a specific equation. The switching may happen for another period and replace the equation by another one adapted for this new regime.

The limit of the MS model that we will avoid here is that regular time series models consider that one set of initial parameters are enough to explain the behavior of the whole data series over time. However, this assumption isn’t always acceptable.

It is known that real-world time series data have different characteristics i.e. means and variances, across different time periods. Therefore, we need to deal with it, and this is what regime-switching models allow. They:

- Characterize data as falling into different, recurring “regimes” or “states”.
- Allow the characteristics of time series data, (including means and variances) and model parameters to change across regimes.
- Assume that, at any given time period, there is a probability that the series may be in any of the regimes and may transition to a different regime.

The aim of the model is to calculate the probability of regime-switching at any given point of the time series. This way, we will better catch the real-world data behaviour than linear and / or regular non-linear models.

4.1 Yahoo Finance

A Python library can be used to extract data from Yahoo Finance using the `yfinance.download()` function. This function extracts the opening price, daily high, daily low, closing price and volume traded each day for any listed asset. Below is an overview of the data for the following pairs BTC-USD, ETH-USD, XRP-USD and ADA-USD. For the following preliminary analysis, we will focus on the last five years (1827 observations) to gather meaningful daily data. Indeed, the more we go back in time the weaker the crypto adoption. Yahoo Finance did not provide data before November 2017 for other cryptocurrencies than Bitcoin. Therefore, for the MS-GARCH analysis later on, we will focus on the last nine trading years of Bitcoin.

BTC-USD						
[*****100%*****] 1 of 1 completed						
	Open	High	...	Adj Close	Volume	
Date			...			
2018-07-14	6247.500000	6298.189941	...	6276.120117	2923670016	
2018-07-15	6272.700195	6403.459961	...	6359.640137	3285459968	
2018-07-16	6357.009766	6741.750000	...	6741.750000	4725799936	
2018-07-17	6739.649902	7387.240234	...	7321.040039	5961950208	
2018-07-18	7315.319824	7534.990234	...	7370.779785	6103410176	
...	...	...	...	...	...	...
2023-07-10	30172.423828	31026.083984	...	30414.470703	14828209155	
2023-07-11	30417.632812	30788.314453	...	30620.951172	12151839152	
2023-07-12	30622.246094	30959.964844	...	30391.646484	14805659717	
2023-07-13	30387.488281	31814.515625	...	31476.048828	23686079548	
2023-07-14	31477.955078	31556.832031	...	31203.656250	26362503168	

Figure 1: BTC-USD historical data from Yahoo Finance

The code corresponding to this section can be found in the Python file `statistics.py`, which you can find in appendix. This file includes a class called 'Statistics' in which we have created functions to extract each of the elements mentioned above so that they can be used independently at a later stage. Other functions calculate returns, logarithmic returns, variance, volatility, skewness, kurtosis and, last but not least, autocorrelations of logarithmic returns as well as autocorrelations of absolute values of logarithmic returns.

First, you will find below a graphic plotting the assets' prices and a graphic plotting their respective returns.

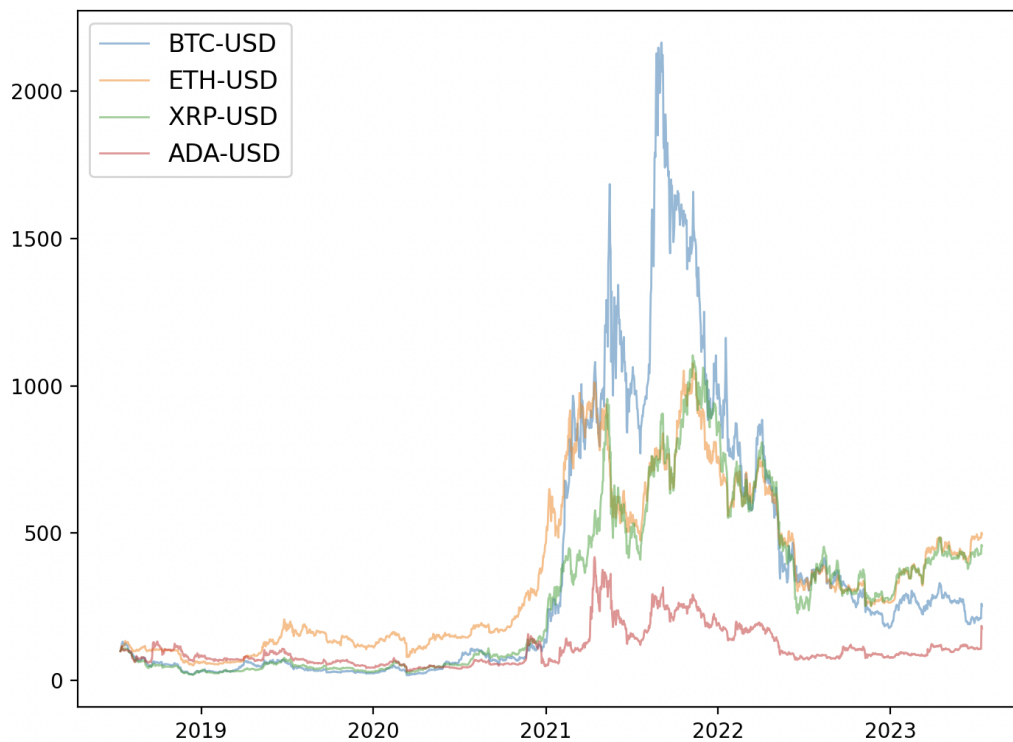


Figure 2: Graphic of prices on base 100

We observe below that the returns seem to progress around 0.

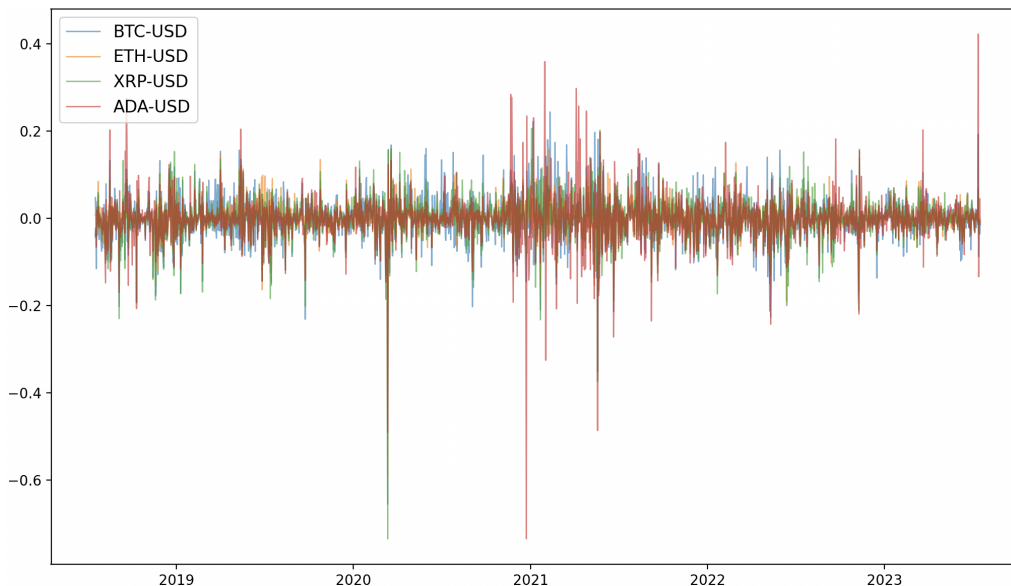


Figure 3: Graphic of returns

4.2 Stationarity tests

Following our study, it is important to check whether our time series are stationary or not. Indeed, the type of models that can be proposed will be significantly different. For instance, a random walk is the simplest non-stationary price process, when an auto-regressive process with mean reversion can be proposed for a stationary process.

A time series (p_t) is weakly stationary if the mean of p_t , the variance of p_t and the covariance between p_t and p_{t+k} do not depend on t for any integer k . We could rather focus on strong stationarity: a time series (p_t) is strongly stationary if, for any integers k, m and any dates $t_1, \dots, t_m$, the law of $(p_{t_1+k}, p_{t_2+k}, \dots, p_{t_m+k})$ does not depend on k . In particular, a random walk $p_t = p_{t-1} + \epsilon_t$, where (ϵ_t) is an independent and identically distributed random sequence (also called “white noise”) is neither strongly or weakly stationary.

The most classical test of stationarity has been proposed by Dickey and Fuller in the 70's. A Dickey-Fuller test assumes that the underlying process can be written

$$p_t = \rho p_{t-1} + \epsilon_t,$$

for some white noise (ϵ_t) and the question is to decide whether $\rho = 1$ (non stationarity, the zero assumption (H0)) or $\rho < 1$ (stationarity, the alternative assumption (H1)). Thus, the Dickey-Fuller test is also called a unit-root test. Here, it will be led at the level 5%: (H0) is not validated when the p-value is less than 5%. In this case, we say that the alternative hypothesis (H1) is validated or that the null hypothesis (H0) is rejected. If the Dickey-Fuller test on a series of asset prices (p_t) does not reject (H0), the latter series has to be modified to become stationary. To this aim, we calculate logarithmic returns: if p_t denotes to price of a crypto asset (the price of 1 BTC in USD, for instance) at time t , then its logarithmic return between $t - 1$ and t is defined as

$$r_t = \ln(p_t/p_{t-1}) \tag{1}$$

or

$$r_t = \ln(p_t) - \ln(p_{t-1}) \tag{2}$$

If the Dickey-Fuller test rejects (H0) on the series of log-returns (r_t), we will consider this series is strongly stationary and it becomes legitimate to assume an underlying MS-GARCH model for (r_t).

You will find below the results of ADF tests for series of prices, returns and logarithmic returns. The more negative the ADF statistic is, the stronger the rejection of the hypothesis that there is a unit root at some level of confidence. P-values are below 5% only in the case where Hypothesis (H1) is validated.

	ADF	p-value
ADA-USD	-1.729459	0.416002
BTC-USD	-1.473130	0.546759
ETH-USD	-1.327966	0.616283
XRP-USD	-2.804397	0.057648

Figure 4: augmented Dickey Fuller test on prices

	ADF	p-value
ADA-USD	-29.382364	0.000000e+00
BTC-USD	-19.730181	0.000000e+00
ETH-USD	-12.626343	1.538682e-23
XRP-USD	-45.046608	0.000000e+00

Figure 5: augmented Dickey Fuller test on returns

	ADF	p-value
ADA-USD	-29.353183	0.000000e+00
BTC-USD	-20.005415	0.000000e+00
ETH-USD	-12.635325	1.474292e-23
XRP-USD	-44.309827	0.000000e+00

Figure 6: augmented Dickey Fuller test on logarithmic returns

Let's see below a graphic with logarithmic returns. We can see that the logarithmic returns allow the removal of any seasonality or trend from the time series.

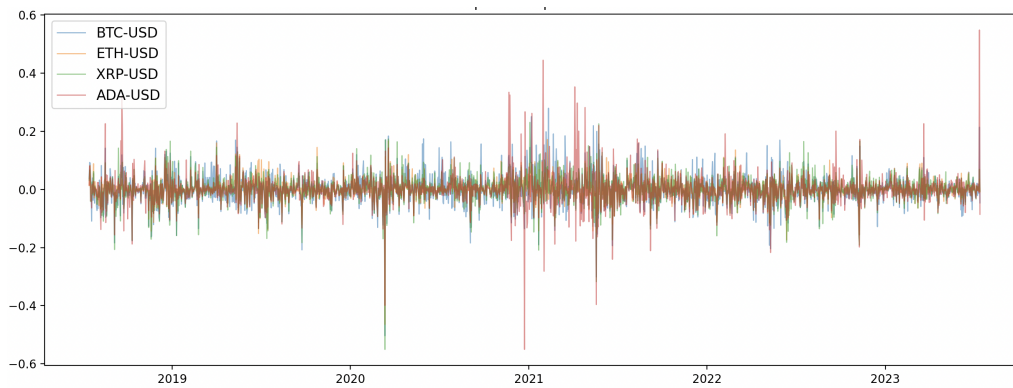


Figure 7: Graphic of logarithmic returns

4.3 Introduction of regime changes

The next stage is to compute the variance and volatility of logarithmic returns of our assets.

	Variance	Volatility
ADA-USD	0.002945	0.86
BTC-USD	0.001331	0.58
ETH-USD	0.002291	0.76
XRP-USD	0.003232	0.90

Figure 8: Table of variances and volatilities for each asset

The variance is a general statistical measure that describes the dispersion of data points around the mean while the volatility describes the variation in the price over time. You will find below how these can help us in demonstrating why studying different regimes is relevant.

We have calculated the average logarithmic returns and the average volatility of the four assets on three periods: an upward period, a flat period and a downward period. The idea is to check the stability (or unstability) of the aforementioned statistics during some particular periods of time.

UP TREND	FLAT TREND	DOWN TREND
27/09/2020 to 08/03/2021	05/01/2020 to 05/07/2020	07/11/2021 to 12/06/2022

Figure 9: Details of up, flat, down periods

You will find below graphics showing BTC-USD price movements within these periods.

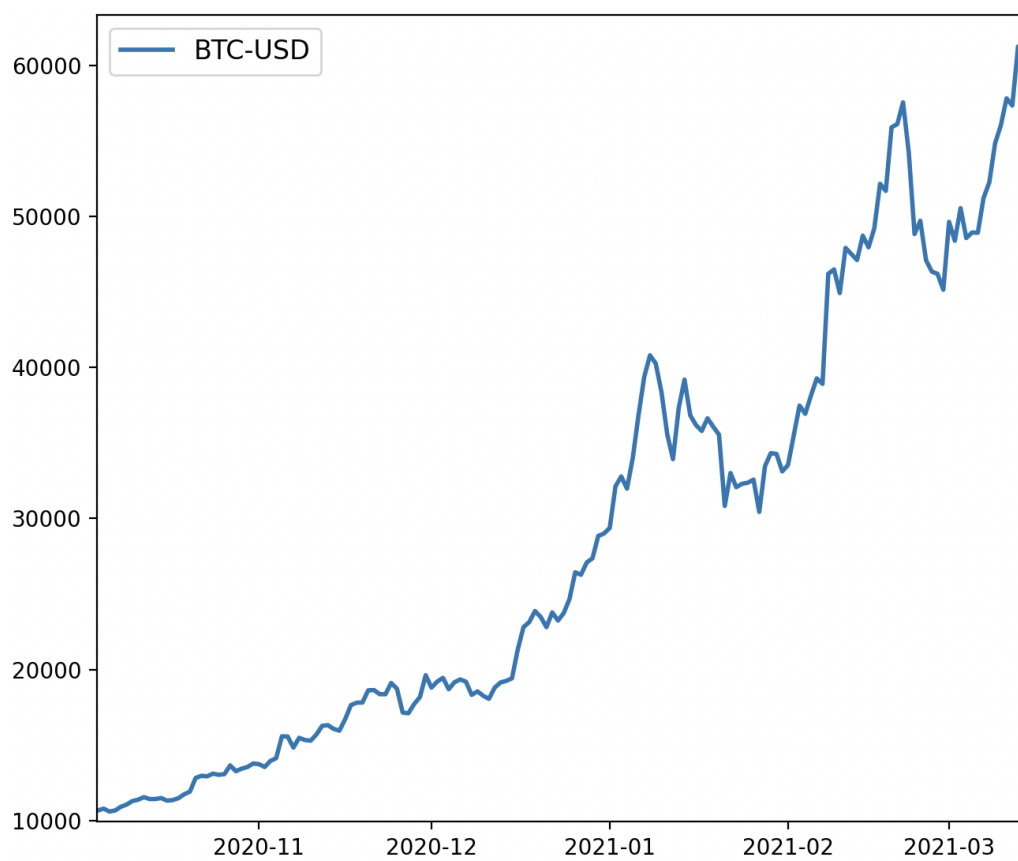


Figure 10: BTC-USD upward trend

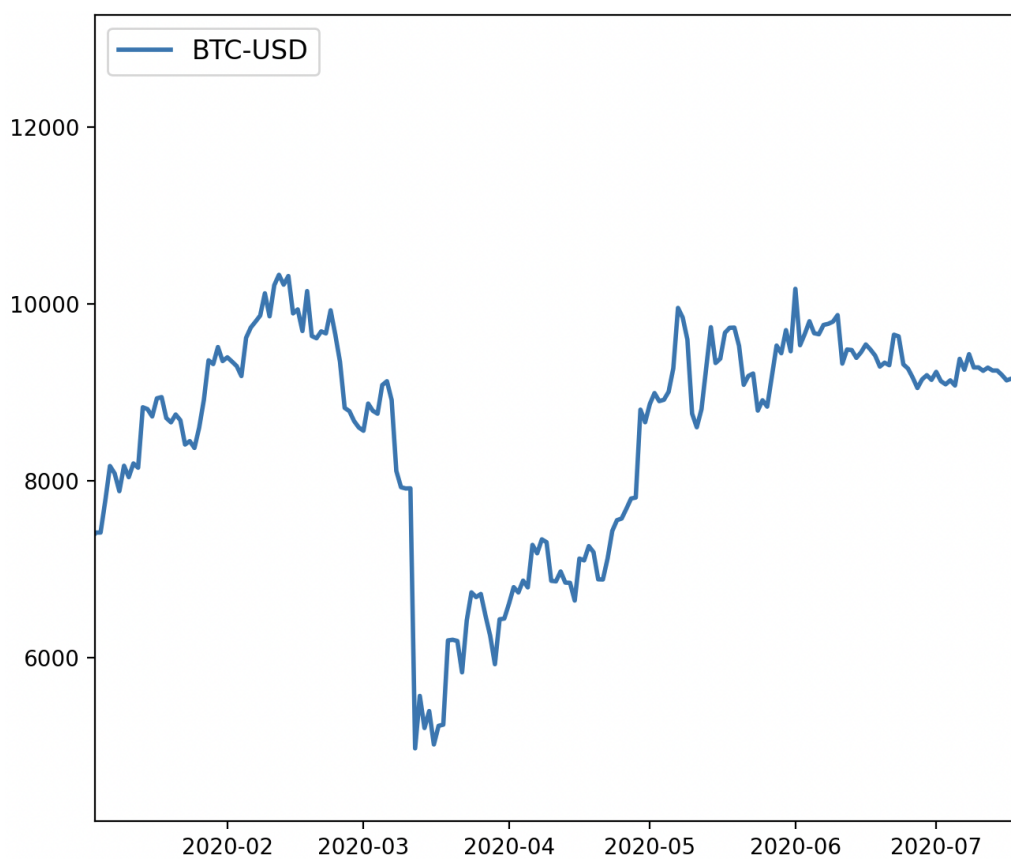


Figure 11: BTC-USD flat trend

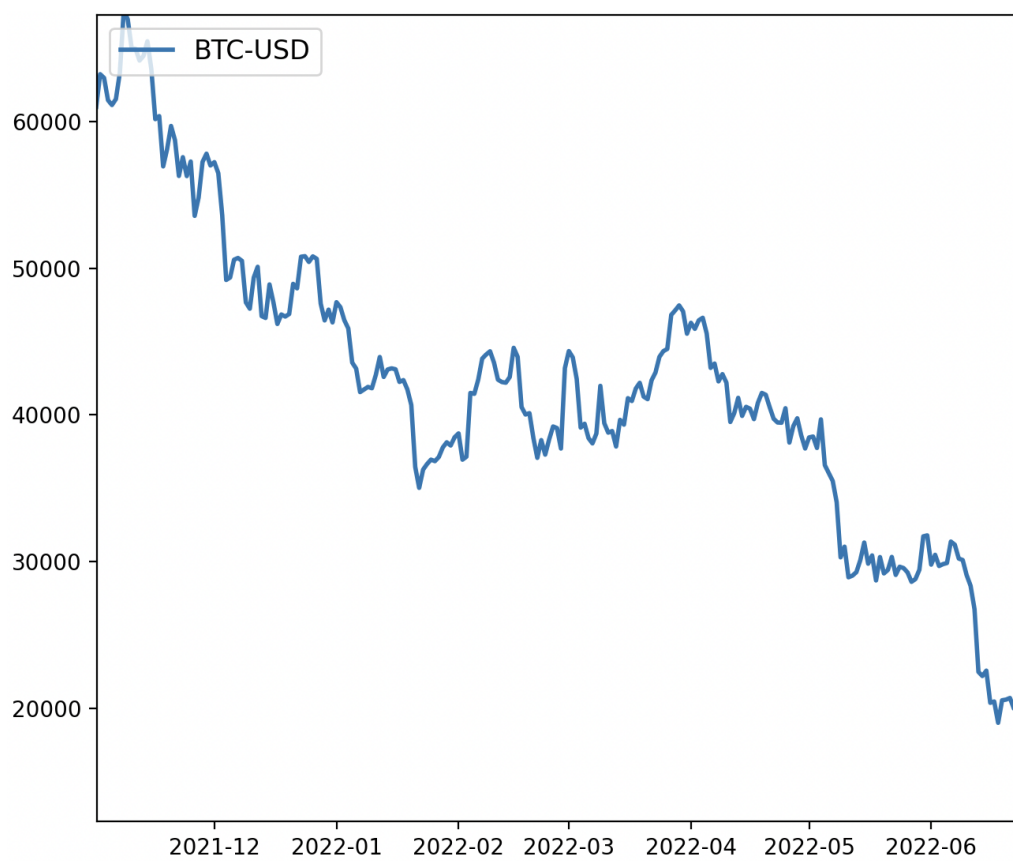


Figure 12: BTC-USD downward trend

We can clearly see in the tables below that average logarithmic returns and average volatilities are highly impacted by the trend of each period. This shows that studying GARCH model into an assumption of regime changes using Markov switching may be highly worthwhile.

BTC-USD	Up	Flat	Down
avg log	0.97%	0.12%	-0.37%
avg vol	64%	78%	54%

Figure 13: BTC-USD average log returns & average volatility per period

ETH-USD	Up	Flat	Down
avg log	0.98%	0.29%	-0.51%
avg vol	83%	99%	65%

Figure 14: ETH-USD average log returns & average volatility per period

XRP-USD	Up	Flat	Down
avg log	0.41%	-0.05%	-0.57%
avg vol	150%	76%	72%

Figure 15: XRP-USD average log returns & average volatility per period

ADA-USD	Up	Flat	Down
avg log	1.5%	0.58%	-0.6%
avg vol	111%	103%	88%

Figure 16: ADA-USD average log returns & average volatility per period

4.4 Skewness & kurtosis

The next statistic to check is skewness and kurtosis which will give valuable insights into the shape of the logarithmic returns distribution. Skewness tells us whether the data is symmetrically distributed around the mean or if it is skewed to one side. On the other hand, Kurtosis tells us whether the data has more outliers (fat tails) or if it is concentrated around the mean (thin tails). Moreover, Skewness and kurtosis help assess the risk and potential returns of investments. Positive skewness may indicate potential gains but also a higher risk of extreme positive returns, while negative skewness may suggest a higher risk of extreme negative returns.

	Skewness	Kurtosis
ADA-USD	-0.227062	6.391967
BTC-USD	-1.155789	16.853523
ETH-USD	-1.129233	12.644640
XRP-USD	0.550580	18.593808

Figure 17: Skewness and kurtosis for each asset

We can see here that the skewness statistics are negative for all, except for Ripple, which is very close to 0. These indicate a relatively higher probability of observing negative log-returns compared to positive log-returns. By examining skewness, we can understand the departure from a normal distribution and assess the degree of asymmetry in the data. The asymmetry remains low as differencing logarithmic returns were meant to reduce trends as much as possible.

We can see here that kurtosis are larger than six, it indicates heavy tailed distribution of the data. Kurtosis helps us identify the probability of extreme values and determine if there exist significant deviations from a normal distribution.

4.5 Autocorrelations

The last very important preliminary analysis that matters is autocorrelations. You will find below two graphics: one for autocorrelations of logarithmic returns and one for autocorrelations of absolute values of BTC-USD logarithmic returns.

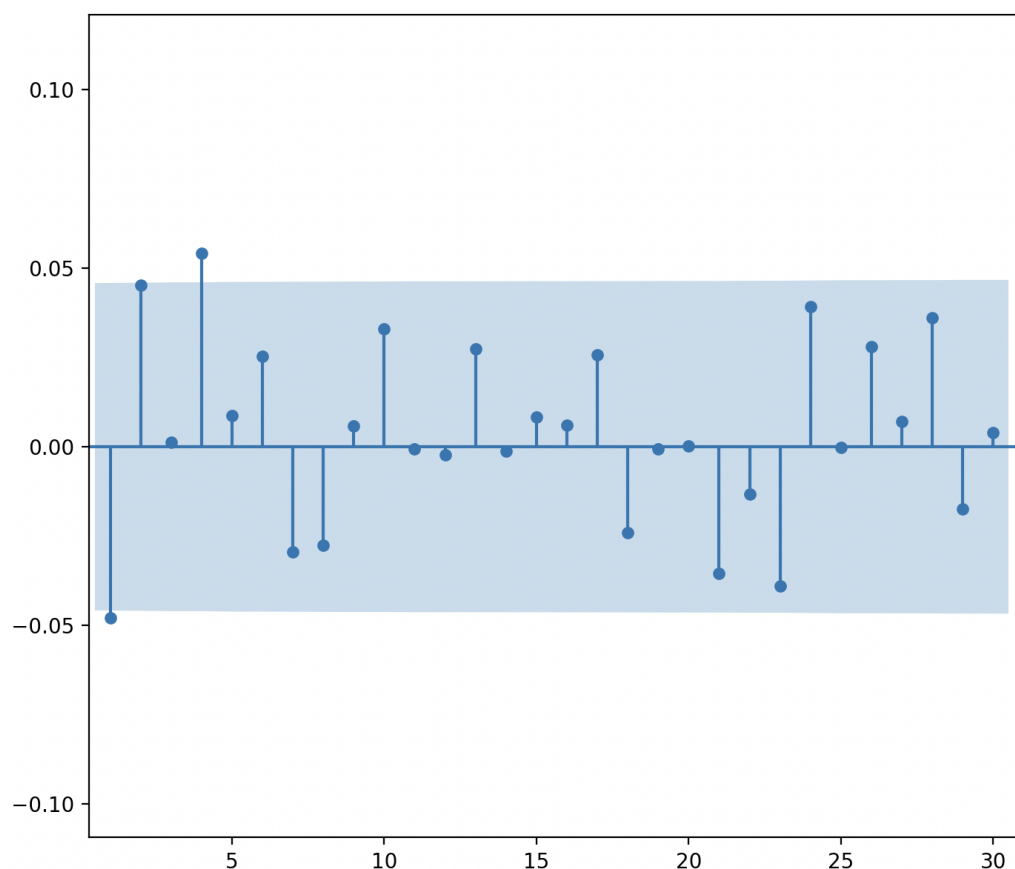


Figure 18: Autocorrelations of BTC-USD logarithmic returns (lags=30)

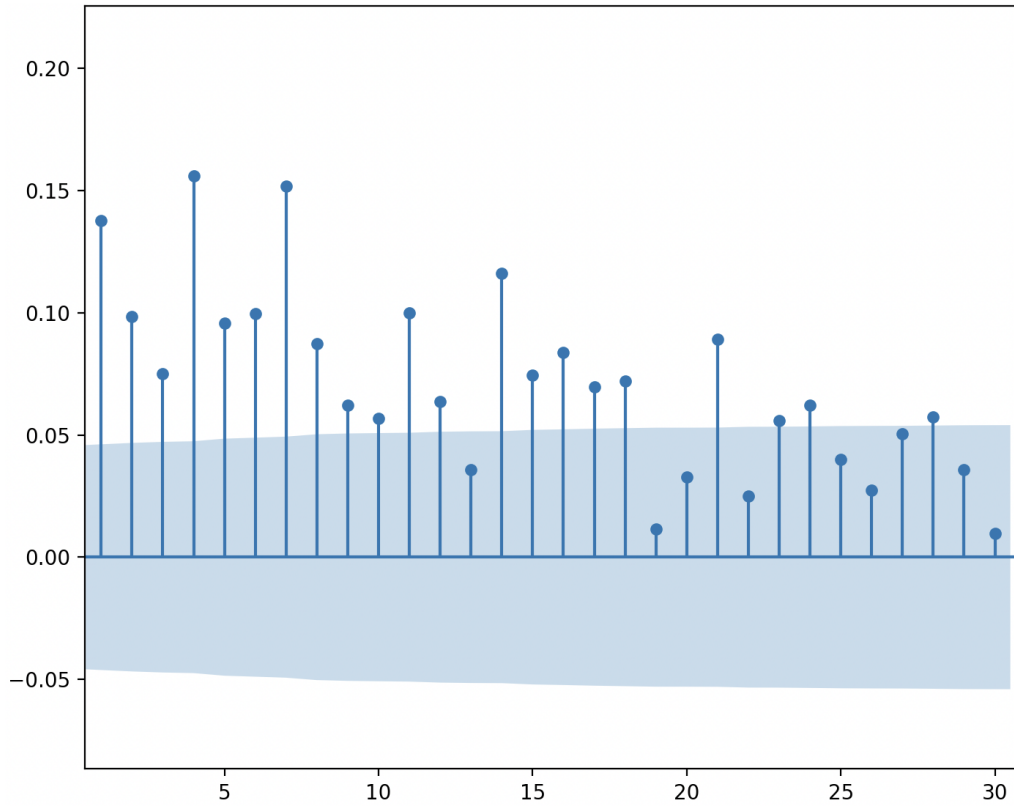


Figure 19: Autocorrelations of absolute values of BTC-USD logarithmic returns (lags=30)

Concerning the autocorrelations of log-returns (Figure 25), almost all of them can be assumed to be zero (with a probability of 95%). A naive analysis would conclude that such returns could be assumed to be independent. This is not the case, as revealed by the autocorrelation of absolute log-returns (Figure 25). Indeed, almost all of them can be considered as nonzero, up to the 20th lag, and they only slowly decrease to zero. If the BTC-USD log-returns were independent, such autocorrelations were close to zero. In particular, this proves that log-returns do not follow a random walk process. Therefore, it makes sense to propose more or less sophisticated models for this series of log-returns.

5 MS-GARCH

5.1 The Model

Since the introduction of Markov-Switching models for the econometric analysis of economic cycles by Hamilton [11], this family of models has been extensively studied in the academic literature and among practitioners. They are based on the intuitive idea that it exists different “states of the world / economy / financial markets” that induce different dynamics for economic and / or financial variables. The switches between the latter states are random and driven by a discrete Markov chain that is not observable (also said latent or hidden, equivalently). Dealing with some parametric models means that these parameters are time-dependent, and their current value at any time t depends on the state of the world s_t at that date.

In other words, a Markov-Switching parametric model for some time series $(X_t)_{t=1,\dots,T}$ is described by a family of distributions (data generating processes) $\{P_\theta, \theta \in \Theta\}$, and by a Markov chain (s_t) that can take K values. When the Markov chain is homogeneous (the typical situation), the law of this Markov chain is deduced from a $K \times K$ transition matrix M , so that

$$M = [p_{ij}], p_{i,j} = \mathbb{P}(s_t = j \mid s_{t-1} = i), (i, j) \in \{1, \dots, K\}^2.$$

Thus, the law of X_t given the current information $\mathcal{F}_{t-1}$ will be P_{θ_t} , where the current parameter θ_t will depend on s_t and on $\mathcal{F}_{t-1}$ (that contains the past values $X_{t-1}, \dots, X_1$ plus, possibly, some additional exogenous information). Note that the dynamics of the successive states (s_t) does not depend on the innovations of the process (X_t) , when the conditional law of X_t depends on s_t . For example, in an economy, assume two underlying states $\{1, 2\}$ (i.e. $K = 2$) called “growth” and “recession”. The dynamics of a vector of macro-economic variables X_t may be set as the vectorial auto-regressive switching regime model

$$X_t = A_{s_t} + B_{s_t}X_{t-1} + \epsilon_t, \epsilon_t \sim \mathcal{N}(0, \Sigma_{s_t})$$

for some vectors (resp. matrices) A_1 and A_2 (resp. B_1 and B_2) and some covariance matrices Σ_1 and Σ_2 .

Markov switching regime models have to be distinguished from mix-

ture models. In the latter case, the law of X_t given $\mathcal{F}_{t-1}$ is randomly drawn among a set of candidates, but independently from one date to the next. Formally, this family of models can be seen as a particular case of Markov-Switching models with a very special transition matrix M for which $\mathbb{P}(s_t = j | s_{t-1} = i) = \mathbb{P}(s_t = j)$ for every (i, j) . In other words, the latter Markov chain has no memory. Since this feature has strong consequences in terms of probabilistic and statistical properties, the two classes of models are distinguished in the literature. See [8] for a very complete overview of these two competing perspectives.

The natural idea of applying switching regime models in financial econometrics rapidly appeared but was not so straightforward. Indeed, asset return distributions exhibit well-known and peculiar empirical features: fat tails (extreme events are significantly more frequent than with Gaussian distributions), skewness (due to the “leverage effect”, the probability of observing large negative returns is typically more frequent than for positive returns), and particularly volatility clustering (some periods of high volatility are followed by some periods of low volatility, due to some “herding behaviors” of dealers in financial markets). Since the latter feature is traditionally captured by GARCH-type models, some authors have proposed Markov-Switching GARCH (MS-GARCH, to be short) models to combine the idea of different regimes/cycles and the persistence of return volatilities. The most natural model specification is then as follows: assume that some series of financial return $(r_t)_{t=1,\dots,T}$ follows the dynamics

$$r_t = \mu_{s_t} + \sigma_t \epsilon_t, \quad (3)$$

where

- the innovations/noises (ϵ_t) is a sequence of independent and identically distributed random variables. Their means are zero and their variances are equal to one.
- (s_t) follows a discrete Markov chain and this process is independent of the noises (ϵ_t) . We assume there will be K underlying states, denoted $1, 2, \dots, K$;
- there are K values $\mu_1, \dots, \mu_K$ for the conditional means;

- the process of conditional volatilities satisfies

$$\sigma_t^2 = \omega_k + a_k \nu_{t-1}^2 + b_k \sigma_{t-1}^2, \text{ when } s_t = k, k \in \{1, \dots, K\}, \quad (4)$$

and ν_t denotes the “unexpected change in the asset return at time t ”. Typically, it can be assumed $\nu_t = r_t - \mu_{s_t}$ or $\nu_t = r_t - \mathbb{E}_{t-1}[\mu_{s_t}]$.

Obviously, (4) refers to the “usual” GARCH(1,1) model, but other more sophisticated and richer dynamics can be introduced there: asymmetries, thresholds, numerous lags, etc. Unfortunately, with all such specifications, the conditional variance σ_t depends not just on the current regime s_t but on the entire past history of the process until $t - 1$, inducing complexities in terms of inference and forecasting. In particular, the calculation of the likelihood becomes numerically untractable. Indeed, the conditional density of any observation r_t given the past depends on all $\sigma_{k,t'}, t' \leq t$ and then all past observations. The first attempts to solve such “path dependencies” were due to [9] and [14], but at the price of questionable and rather ad-hoc modifications of Equation (4). See Appendix 5.3 for details.

A more convenient and sounder proposition has been made by [10]. They assumed the existence of K “parallel” dynamics

$$\sigma_{k,t}^2 = \omega_k + a_k r_{t-1}^2 + b_k \sigma_{k,t-1}^2, \text{ when } k \in \{1, \dots, K\}. \quad (5)$$

when the asset return process is $r_t = \sigma_{s_t,t} \epsilon_t$. Note that the conditional expectation of r_t is assumed to be zero here (i.e. no μ_{s_t}). Thus, the latter process does no longer suffers from path dependencies for inference purpose. In particular, the estimation of the model given by (5) and in [10] can be led by maximum likelihood thanks to the use of the Hamilton filter (as for “usual” Markov-Switching regression models, in the spirit of [11]). Alternative methods are available, notably through Gibbs sampling under a Bayesian perspective ([4]), by a mix of EM-algorithm and importance sampling ([3]), or by the General Method of Moments ([7]). A R-package is available for estimating this family of models, with some variants of (5) and several potential laws for ϵ_t .

The dynamics of crypto-asset returns exhibit special features that cannot be easily captured with simple dynamic models. In particular, many authors suggest the existence of several regimes, particularly three: “boom”, “bust” and “nothing special”. This has fuelled a burgeoning empirical liter-

ature that rapidly expands. In particular, some authors have logically tested MS-GARCH models as discussed above: see [2, 1, 6, 15], among others. Most papers insist on the relevance of considering several regimes to describe volatility dynamics. Nonetheless, to the best of our knowledge, all of them assume that the conditional means μ_{s_t} are zero, mainly for convenience and to avoid the introduction of path dependencies. However, it is relatively easy to modify the methodology of [10] by assuming $r_t = \mu_{s_t} + \sigma_{s_t,t}\epsilon_t$ and

$$\sigma_{k,t}^2 = \omega_k + a_k(r_{t-1} - \mu_k)^2 + b_k\sigma_{k,t-1}^2, \text{ for any } k \in \{1, \dots, K\}. \quad (6)$$

This extension of the core model of [10] had been mentioned by these authors in passing (Section 2.3.2 in [10]). Since the crypto assets seem to exhibit momentums, the possibility of non zero conditional means becomes a key issue that has to be checked.

Therefore, we hereafter extend the methodology of [10] to manage non-zero conditional means. In particular, the chosen estimation procedure will be the maximum likelihood method. Since the corresponding code does not seem to be available on the web, this project will require coding our inference method.

5.2 Calculation of a likelihood function

We observe a sequence of asset returns $r_1, \dots, r_T$. Denote by $\mathcal{F}_t$ the observations until time t , i.e. $r_1, \dots, r_t$. Assume the underlying MS-GARCH model is given by

$$r_t = \mu_{s_t} + \sigma_{s_t,t}\epsilon_t, \quad \mathbb{E}[\epsilon_t | \mathcal{F}_{t-1}] = 0, \quad \mathbb{E}[\epsilon_t^2 | \mathcal{F}_{t-1}] = 1, \quad (7)$$

where

$$\sigma_{k,t}^2 = \omega_k + a_k(r_{t-1} - \mu_k)^2 + b_k\sigma_{k,t-1}^2, \quad (8)$$

for any $k \in \{1, \dots, K\}$. The innovations $\epsilon_1, \dots, \epsilon_T$ are i.i.d. The density of ϵ_t is denoted f_ϵ .

Denote $p_{i,j} := \mathbb{P}(s_t = j | s_{t-1} = i)$, when $i, j \in \{1, \dots, K\}$ the unknown transition probabilities. The vector of parameters is denoted as θ . It stacks all subvectors (ω_k, a_k, b_k) , $k \in \{1, \dots, K\}$, the vector of conditional means $(\mu_1, \dots, \mu_K)$, the unknown probabilities $p_{i,j}$ when $i < j$ and, possibly, some additional parameters that define the law of ϵ_t . Thus, given the past until

time $t - 1$, i.e. given $\mathcal{F}_{t-1}$, the law of the asset return at time t depends on the state s_t . The latter state is unobservable and determines the conditional mean μ_{s_t} of r_t and its conditional variance σ_t (Equation (7)).

The conditional density of r_t given its the past values can be written

$$\begin{aligned}
f(r_t|\mathcal{F}_{t-1}) &= \sum_{i,j=1}^K f(r_t, s_t = j, s_{t-1} = i|\mathcal{F}_{t-1}) \\
&= \sum_{i,j=1}^K f(r_t|s_t = j, s_{t-1} = i, \mathcal{F}_{t-1})\mathbb{P}(s_t = j, s_{t-1} = i|\mathcal{F}_{t-1}) \\
&= \sum_{i,j=1}^K p_{i,j}f(r_t|s_t = j, \mathcal{F}_{t-1})\mathbb{P}(s_{t-1} = i|\mathcal{F}_{t-1}), \tag{9}
\end{aligned}$$

when $t \in \{1, \dots, T\}$. We will set $\mathcal{F}_0 = \emptyset$ and $f(r_1|\mathcal{F}_0) = f(r_1)$, where f denotes the unconditional distribution of r_1 . In other words,

$$f(r_1|\mathcal{F}_0) = f_\epsilon((r_1 - \mathbb{E}[\mu_{s_t}])/\mathbb{E}[\sigma_{s_t,0}])/\mathbb{E}[\sigma_{s_t,0}].$$

In practice, we will simply replace $\mathbb{E}[\mu_{s_t}]$ (respectively $\mathbb{E}[\sigma_{s_t,0}]$) by the empirical mean (resp. standard deviation) of (r_t) .

The goal is to calculate (9) for every $t \in \{1, \dots, T\}$, and then the log-likelihood. Indeed, the log-likelihood of the sample will be

$$\mathcal{L}(\theta) := \sum_{t=1}^T \ln f(r_t|\mathcal{F}_{t-1}),$$

and the latter function has to be optimized with respect to its parameter θ .

First, we evaluate $f(r_t|s_t = j, \mathcal{F}_{t-1})$ through the law of ϵ_t , once $\sigma_{j,t}$ is evaluated. This latter point is done by recursively using (8) backwards, from t to the initial date. To be more specific, denote by f_ϵ the density of ϵ_t . Thus, we have

$$f(r_t|s_t = j, \mathcal{F}_{t-1}) = f_\epsilon((r_t - \mu_j)/\sigma_{j,t})/\sigma_{j,t}, \tag{10}$$

and the calculation of $\sigma_{j,t}$ can be made iteratively:

$$\begin{aligned}
\sigma_{j,t}^2 &= \omega_j + a_j(r_{t-1} - \mu_j)^2 + b_j\sigma_{j,t-1}^2 \\
&= \omega_j(1 + b_j) + a_j(r_{t-1} - \mu_j)^2 + b_j a_j(r_{t-2} - \mu_j)^2 + b_j^2 \sigma_{j,t-2}^2 \\
&= \frac{\omega_j(1 - b_j^t)}{1 - b_j} + \sum_{k=1}^t a_j b_j^{k-1} (r_{t-k} - \mu_j)^2 + b_j^t \sigma_{j,0}^2.
\end{aligned}$$

To initialize, set the quantities $\sigma_{j,0}$, $j \in \{1, \dots, K\}$ such that

$$\sigma_{1,0} < \sigma_{2,0} < \dots < \sigma_{0,K}.$$

In practice, these values will be distributed around the unconditional standard deviation σ_0 of (r_t) (i.e. $\mathbb{E}[r_t^2] = \sigma_0^2$), and σ_0 will be estimated by the empirical standard deviation $\hat{\sigma}_0$ of the sample. For instance, if $K = 2$, choose $(\sigma_{1,0}, \sigma_{2,0})$ s.t. $\sigma_{1,0} < \hat{\sigma}_0 < \sigma_{2,0}$. If $K = 3$, choose $(\sigma_{1,0}, \sigma_{2,0}, \sigma_{3,0})$ s.t. $\sigma_{1,0} < \sigma_{2,0} = \hat{\sigma}_0 < \sigma_{3,0}$.

In (9), the quantity $\mathbb{P}(s_{t-1} = i | \mathcal{F}_{t-1})$ can be calculated by the Hamilton filter (see [13], Chapter 22), that we recall: introduce the notations $\zeta_{i,t|t} := \mathbb{P}(s_t = i | \mathcal{F}_t)$ and $\zeta_{i,t|t-1} := \mathbb{P}(s_t = i | \mathcal{F}_{t-1})$. Thus, we have

$$\begin{aligned}
\zeta_{i,t|t} &= \mathbb{P}(s_t = i | \mathcal{F}_t) = \frac{\mathbb{P}(s_t = i, r_t | \mathcal{F}_{t-1})}{f(r_t | \mathcal{F}_{t-1})} \\
&= \frac{f(r_t | s_t = i, \mathcal{F}_{t-1}) \mathbb{P}(s_t = i | \mathcal{F}_{t-1})}{\sum_{k=1}^K f(r_t | s_t = k, \mathcal{F}_{t-1}) \mathbb{P}(s_t = k | \mathcal{F}_{t-1})} \\
&= \frac{f(r_t | s_t = i, \mathcal{F}_{t-1}) \zeta_{i,t|t-1}}{\sum_{k=1}^K f(r_t | s_t = k, \mathcal{F}_{t-1}) \zeta_{k,t|t-1}}. \tag{11}
\end{aligned}$$

But, note that

$$\zeta_{k,t|t-1} = \sum_{l=1}^K p_{kl} \zeta_{l,t-1|t-1},$$

and $f(r_t | s_t = i, \mathcal{F}_{t-1})$ can be calculated as in (10). This yields

$$\zeta_{i,t|t} = \mathbb{P}(s_t = i | \mathcal{F}_t) = \frac{f(r_t | s_t = i, \mathcal{F}_{t-1}) \sum_{l=1}^K p_{kl} \zeta_{l,t-1|t-1}}{\sum_{k,l=1}^K f(r_t | s_t = k, \mathcal{F}_{t-1}) p_{kl} \zeta_{l,t-1|t-1}}. \tag{12}$$

Finally, (11) allows to recursively calculate the quantities $\zeta_{i,t|t}$, and then the conditional density of r_t given its the past values.

The estimator of θ is then

$$\hat{\theta} := \arg \max_{\theta \in \Theta} \mathcal{L}(\theta) = \arg \max_{\theta \in \Theta} \sum_{t=1}^T \ln f(r_t | \mathcal{F}_{t-1}), \quad (13)$$

for some subset closed and bounded Θ .

In practice, the convenient methodology is as follows:

- code a routine for the density of the innovations, i.e. the maps f_ϵ . We will first choose the density of a Gaussian random variable $\mathcal{N}(0, 1)$. Afterwards, we will see whether a Student law or even a more complex one is a good idea. But leave open the possibility of having one or several unknown parameters that are related to the law of the innovations.
- code the calculation of the quantities $\zeta_{i,t|t}$ and $f(r_t | s_t = j, \mathcal{F}_{t-1})$. They depend on each other and have to be considered simultaneously.
- build a procedure for the calculation of the log-likelihood $\theta \mapsto \mathcal{L}(\theta)$.
- invoke an Python optimizer to solve (13).

It is common practice to lead a “sanity check” by simulation to check that the latter code works:

1. select a value for θ and the true Data Generating Process is given by (7);
2. generate a sequence $(r_1, \dots, r_T)$, $T = 1000$;
3. calculate $\hat{\theta}$ and compare its value with the true parameter.

5.3 Path dependencies in MS-GARCH models

In this section, we explain the difficulty of estimating a Markov-Switching GARCH model by maximum likelihood (the main universal estimation procedure), starting from the realization of a time series $(z_t)_{t=1, \dots, T}$.

To fix the ideas, first consider the simplest Markov-Switching model as possible: there exists an underlying homogenous Markov chain (s_t) , $s_t \in$

$\{1, 2\}$ and $z_t \sim \mathcal{N}(\mu_{s_t}, 1)$, for every $t \in \{1, \dots, T\}$. The parameter θ to be estimated is the concatenation of μ_1, μ_2 and two transition probabilities $p_{1,2}$ and $p_{2,1}$. The log-likelihood associated to the observed path and the latter model is $\mathcal{L}(\theta) = \sum_{t=1}^T \ln f(z_t|z_{t-1}, \dots, z_1)$ where $f(z_t|z_{t-1}, \dots, z_1)$ is the conditional density of z_t given its past values. Since s_t is unknown, we have

$$\begin{aligned} f(z_t|z_{t-1}, \dots, z_1) &= \sum_{j=1}^2 \mathbb{P}(s_t = j|z_{t-1}, \dots, z_1) f(z_t|s_t = j, z_{t-1}, \dots, z_1) \\ &= \sum_{j=1}^2 \mathbb{P}(s_t = j|z_{t-1}, \dots, z_1) \phi(z_t - j), \end{aligned}$$

denoting by ϕ the density of a standard Gaussian random variable: $\phi(t) = \exp(-t^2/2)/\sqrt{2\pi}$, for every t .

By the so-called Hamilton filter, it is possible to explicitly and iteratively calculate the quantities $\mathbb{P}(s_t = j|z_{t-1}, \dots, z_1)$. Thus, the calculation of $\mathcal{L}(\theta)$ can effectively be made; in particular, the latter map can easily be optimized with respect to θ , to obtain the maximum likelihood estimator of the model parameter.

In the case of MS-GARCH models, for which the current volatility at time t depends on the states s_t and on the whole path $(z_j)_{j \leq t}$, things become more complex. Now, assume a very simple MS-GARCH model: for every t , $z_t \sim \mathcal{N}(0, \sigma_{t,s_t}^2)$ with the variance dynamics

$$\sigma_{t,j}^2 = \omega_j + a_j r_{t-1}^2 + b_j \sigma_{t-1,s_{t-1}}^2, \quad j \in \{1, 2\}. \quad (14)$$

The new vector of parameters becomes

$$\theta := (\omega_1, \omega_2, a_1, a_2, b_1, b_2, p_{1,2}, p_{2,1}).$$

The calculation of the associated log-likelihood, or, equivalently, of the conditional densities $f(z_t|z_{t-1}, \dots, z_1)$ will induce the so-called “path dependency problem”. Indeed, $f(z_t|z_{t-1}, \dots, z_1)$ depends on σ_{t,s_t} , that is not observable and depends itself on all the past states (not only s_t). As a consequence, we

have

$$f(z_t|z_{t-1}, \dots, z_1) = \sum_{s_1=1}^2 \cdots \sum_{j_t=1}^2 \mathbb{P}(s_1 = j_1, \dots, s_t = j_t | z_{t-1}, \dots, z_1) \\ f(z_t | s_1 = j_1, \dots, s_t = j_t, z_{t-1}, \dots, z_1).$$

The latter formula cannot be calculated analytically in practice because it involves 2^T terms, i.e. an exploding number of terms. Therefore, we are facing a problem of computational statistics ¹.

The first attempts to solve such “path dependencies” were due to [5] and [12]. The latter authors introduced some regime-switching models in which the conditional variances in each regime depend on the a finite and reduced number of past variances and squared returns. This means considering ARCH proceses instead of GARCH processes, and restricting “memory” to a finite number of past periods. This is clearly restrictive for financial returns. Indeed, it is widely recognized that they exhibit long-memory patterns.

Another proposition was due to [9] that apparently slightly modified the MS-GARCH dynamics to circumvent path dependencies. In his paper, he replaced (14) by

$$\sigma_{t,j}^2 = \omega_j + a_j r_{t-1}^2 + b_j \{p_{1,t-1} \sigma_{t-1,1}^2 + p_{2,t-1} \sigma_{t-1,2}^2\}, \text{ where} \quad (15)$$

$$p_{j,t-1} = \mathbb{P}(s_{t-1} = j | z_{t-2}, \dots, z_1), \quad j \in \{1, 2\}.$$

The quantities $p_{j,t-1}$ can be iteratively calculated by Hamilton filter. Under the new dynamics (15), each conditional variance depends only on the current regime, not on the entire past history of the process, breaking the path dependency curse. The associated tree is said to be “recombining”. Note that the term $p_{1,t-1} \sigma_{t-1,1}^2 + p_{2,t-1} \sigma_{t-1,2}^2$ is the conditional expectation of $\sigma_{t-1,s_{t-1}}^2$ given the information until $t-2$ in terms of returns and states (or, equivalently given r_{t-2} because (s_t) is an homogenous Markov chain).

Alternatively, [14] promoted the same type of model as [9], but by modi-

¹This is explained slightly differently in [9]: “The conditional variance at time t depends upon the conditional variance at time $t-1$, which depends upon the regime at time $t-1$ and on the conditional variance at time $t-2$, and so on. Consequently, the conditional variance at time t depends on the entire sequence of regimes up to time t . The likelihood function is constructed by integrating over all possible paths”.

fying the way of calculating the average variances in the updating equations:

$$\sigma_{t,j}^2 = \omega_j + a_j r_{t-1}^2 + b_j \{q_{1,t-1} \sigma_{t-1,1}^2 + q_{2,t-1} \sigma_{t-1,2}^2\}, \text{ where} \quad (16)$$

$$q_{j,t-1} = \mathbb{P}(s_{t-1} = j | s_t = j, z_{t-1}, \dots, z_1), \quad j \in \{1, 2\}.$$

Note the difference between $q_{j,t-1}$ and the previous quantity $p_{j,t-1}$. Thus, [14] uses more information for the purpose of average variance calculations, say the knowledge of the current state s_t and, in addition, the most recent observed return z_{t-1} . He considers that “this extra data embodies information about previous regimes and is thus useful”.

In [9] and [14], the path dependency problem is apparently solved, but at the price of questionable and rather ad-hoc modifications of Equation (14). These modifications may not be justified by economic and/or financial reasonings, or intuitive modeling intuition. They should rather seen as some technical tricks to avoid the numerical difficulty of maximum likelihood inference. This justifies the introduction of the more realistic and more intuitive Markov-Switching GARCH model of [10] that we study and extend in this report.

6 Results

At last, we go down to results. Our goal was to improve the MS-GARCH model for the return of BTC in USD, by using different conditional means for each regime as previous researchers considered a zero conditional mean. Some empirical features (see Section 4.3 and [1]) lead us to assume the existence of three hidden states. Thus, hereafter, we set $K = 3$.

The code getting to the following results can be found in appendix of this Master's thesis.

6.1 Parameters & conditional means

The first output from our code is a table of parameters for each regime that you can see below in figure 20.

<i>Parameter</i>	<i>Regime 1</i>	<i>Regime 2</i>	<i>Regime 3</i>
<i>mu</i>	-0.00123	0.00062	0.00118
<i>omega</i>	0.00080	0.00028	0.00003
<i>alpha</i>	0.14895	0.13465	0.09025
<i>beta</i>	0.84796	0.71776	0.74620
<i>gamma</i>	[0.00, -2.91336, -2.13776]	[-4.32279, 0.00, -4.68771]	[-3.48848, -4.77557, 0.00]
<i>P</i>	[0.85309, 0.04632, 0.10060]	[0.01297, 0.97802, 0.00901]	[0.02940, 0.00812, 0.96248]

Figure 20: Output parameters of our MS-GARCH model

mu

As our primary hypothesis stated it, we can see below that conditional means μ_k , $k \in \{1, 2, 3\}$ are very much different from one regime to another. Indeed, the annual average return in regime one ("bear market") is close to -89% ; in regime 2 ("bull market"), this quantity is around 123% . Finally, the third regime ("normal time") is associated with an annual mean return around 22% .

This shows that using a the same conditional mean for each regime, or simplifying even more by assuming all conditional means are zero (the standard way of working in the academic literature, following [10]), inevitably

leads to less accuracy. Thus, the so-called μ_k , $k \in \{1, \dots, K\}$ have to be estimated, leading to more flexible and realistic econometric models.

omega

Our omegas are positive (which is coherent with the model).

alpha

Our alphas are rather small. That means that the volatility of the Bitcoin is not much sensible to chocs.

beta

Our high betas mean that Bitcoin's volatility is highly sensible to trends, much more than chocs.

gamma

Gammas are “instrumental” parameters to calculate the transition probabilities of switching from one regime to another. They allow a practical reparametrization of the transition matrix. Indeed, they can take any real value and they do not have to satisfy some linear constraints. The relationships between the gammas and the transition probabilities $p_{i,j} = \mathbb{P}(s_t = j | s_{t-1} = i)$, $(i, j) \in \{1, \dots, 3\}$, are as follows:

$$p_{1,2} = \frac{\exp(\gamma_{1,2})}{1 + \exp(\gamma_{1,2}) + \exp(\gamma_{1,3})},$$

$$p_{1,3} = \frac{\exp(\gamma_{1,3})}{1 + \exp(\gamma_{1,2}) + \exp(\gamma_{1,3})},$$

and, obviously, $p_{1,1} = 1 - p_{1,2} - p_{1,3}$. And similarly for the other transition probabilities.

The estimated Transition Matrix P

Reading the transition matrix helps understanding the probabilities of switching from a regime to another. For instance, the daily probability of switching from state 1 to state 2 is 0,6%. See the discussion in Section 6.2.

6.2 Regime persistence

Let's now draw the probabilities of being in any regime for each day t , given the prices until $t-1$. We observe that the most persistent regime is the second

one ("bull market"): there exist relatively long periods of BTS-USD price increases. This justifies the applicability of some "momentum" investment strategies based on Bitcoin. The least persistent regime is the "bear market": once $s_t = 1$, the probability of switching to the third regime the day after is very (72%). This may be due to the fact that a price decrease is often followed by a "wait-and-see" market. Such effects could be explained by some microstructure effects on the BTC-USD market, by the interaction between informed dealers and the others, etc., but such discussions go beyond the scope of this work. Globally, our model allows to affirm there are really three meaningful underlying regimes that are relatively short-lived in average, particularly during bear markets.

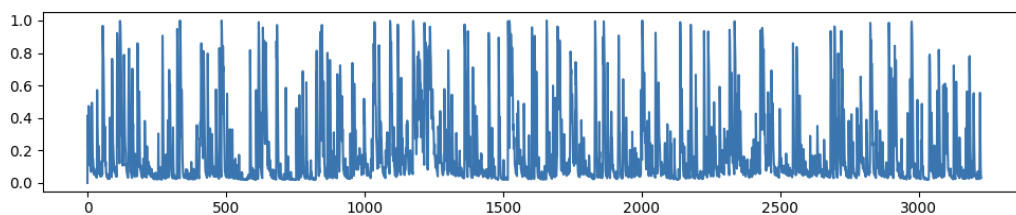


Figure 21: Probabilities of regime 1

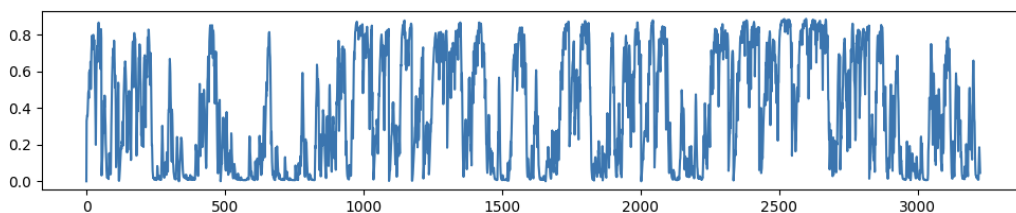


Figure 22: Probabilities of regime 2

You will find in appendix the exact probabilities of being in regime i at date t , knowing the returns until $t-1$:

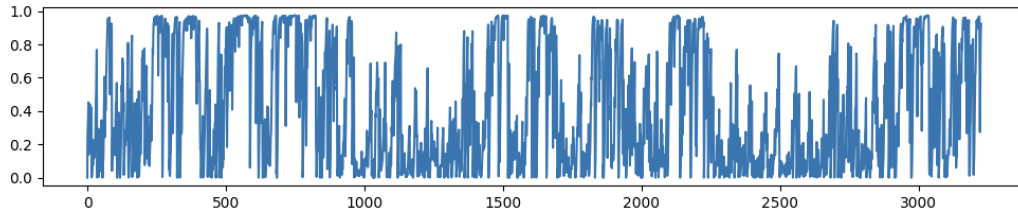


Figure 23: Probabilities of regime 3

6.3 Mean variances per regime

We will now compute the mean variance in each regime with the following formula:

$$\bar{\sigma}_k^2 = \omega_k / (1 - \alpha_k - \beta_k) \quad (17)$$

<i>Mean Variance</i>	<i>Regime 1</i>	<i>Regime 2</i>	<i>Regime 3</i>
$\bar{\sigma}_k^2$	0.00668	0.00168	0.00065

Figure 24: Mean Variance per regime

7 Conclusion

The aims from the start were to implement the MS-GARCH model and check if:

- Multi-regimes models were effectively helpful
- Computing a conditional mean for each regime with path-dependancy improves the MS-GARCH model

Objectives are fulfilled. Subsequently, it could be interesting to run the model on different assets. Yahoo Finance could not provide us data for data before 2017 for other cryptocurrencies than Bitcoin. And Bitcoin itself was only available from September 2014. It would also be interesting to use this model for forecasting prices. But this could be another Master's thesis itself!

Mathematically, the limits are the following: No calculation of confidence intervals for the estimated parameters. We could test other MS-GARCH-type models; typically, we could estimate MS-GARCH models where the laws for the innovations are no longer Gaussian but fat tailed (Student, e.g.), or asymmetrical. Or the updating equation of conditional variances in a MS framework could be different and inspired by the numerous GARCH-type dynamics: EGARCH, TGARCH, GJRARCH, etc. (see [6], e.g.).

8 Bibliography

References

- [1] David Ardia, Keven Bluteau, and Maxime Rüede. “Regime changes in Bitcoin GARCH volatility dynamics”. In: *Finance Research Letters* 29 (2019), pp. 266–271.
- [2] David Ardia et al. “Forecasting risk with Markov-switching GARCH models: A large-scale performance study”. In: *International Journal of Forecasting* 34.4 (2018), pp. 733–747.
- [3] Maciej Augustyniak. “Maximum likelihood estimation of the Markov-switching GARCH model”. In: *Computational Statistics & Data Analysis* 76 (2014), pp. 61–75.
- [4] Luc Bauwens, Arie Preminger, and Jeroen VK Rombouts. “Theory and inference for a Markov switching GARCH model”. In: *The Econometrics Journal* 13.2 (2010), pp. 218–244.
- [5] Jun Cai. “A Markov model of unconditional variance in ARCH”. In: *Journal of Business and Economic Statistics* 12.3 (1994), pp. 309–316.
- [6] Guglielmo Maria Caporale and Timur Zekokh. “Modelling volatility of cryptocurrencies using Markov-Switching GARCH models”. In: *Research in International Business and Finance* 48 (2019), pp. 143–155.
- [7] Christian Francq, Jean-Michel Zakoi, et al. “Deriving the autocovariances of powers of Markov-switching GARCH models, with applications to statistical inference”. In: *Computational Statistics & Data Analysis* 52.6 (2008), pp. 3027–3046.
- [8] Sylvia Frühwirth-Schnatter. *Finite mixture and Markov switching models*. Springer, 2006.
- [9] Stephen F Gray. “Modeling the conditional distribution of interest rates as a regime-switching process”. In: *Journal of Financial Economics* 42.1 (1996), pp. 27–62.
- [10] Markus Haas, Stefan Mittnik, and Marc S Paoletta. “A new approach to Markov-switching GARCH models”. In: *Journal of financial Econometrics* 2.4 (2004), pp. 493–530.

- [11] James D Hamilton. “A new approach to the economic analysis of non-stationary time series and the business cycle”. In: *Econometrica: Journal of the econometric society* (1989), pp. 357–384.
- [12] James D Hamilton and Raul Susmel. “Autoregressive conditional heteroskedasticity and changes in regime”. In: *Journal of econometrics* 64.1-2 (1994), pp. 307–333.
- [13] James Douglas Hamilton. *Time series analysis*. Princeton university press, 2020.
- [14] Franc Klaassen. “Improving GARCH volatility forecasts with regime-switching GARCH”. In: *Advances in Markov-switching models*. Springer, 2002, pp. 223–254.
- [15] Theodore Panagiotidis, Georgios Papapanagiotou, and Thanasis Stengos. “On the volatility of cryptocurrencies”. In: *Research in International Business and Finance* 62 (2022), p. 101724.

9 Appendix

Beware that all the code required for this Master’s thesis is visible on my own github account (username: carlpaulus, repository: msgarch_carl).

<i>Date</i>	<i>Regime 1</i>	<i>Regime 2</i>	<i>Regime 3</i>
20/09/2014	41.60%	32.93%	25.46%
21/09/2014	35.49%	34.68%	29.83%
22/09/2014	28.26%	36.12%	35.61%
23/09/2014	19.17%	35.65%	45.18%
24/09/2014	47.36%	38.89%	13.76%
25/09/2014	36.04%	44.41%	19.55%
26/09/2014	26.17%	48.41%	25.43%
27/09/2014	16.87%	49.07%	34.06%
28/09/2014	10.53%	45.40%	44.07%
29/09/2014	24.48%	59.38%	16.14%
30/09/2014	14.23%	60.15%	25.61%
01/10/2014	11.18%	60.67%	28.15%
02/10/2014	7.10%	54.05%	38.85%
03/10/2014	6.78%	51.18%	42.04%
04/10/2014	13.14%	65.29%	21.56%
...	...	...	...
06/07/2023	3.26%	1.92%	94.81%
07/07/2023	7.15%	3.08%	89.77%
08/07/2023	4.74%	2.55%	92.71%
09/07/2023	2.88%	1.62%	95.50%
10/07/2023	2.56%	1.17%	96.27%
11/07/2023	2.56%	0.98%	96.46%
12/07/2023	2.42%	0.85%	96.73%
13/07/2023	2.69%	0.88%	96.43%
14/07/2023	35.76%	7.62%	56.62%
15/07/2023	55.49%	17.14%	27.38%
16/07/2023	25.61%	18.57%	55.82%
17/07/2023	9.40%	13.51%	77.09%
18/07/2023	4.34%	8.10%	87.56%
19/07/2023	3.07%	4.61%	92.32%
08/07/2023	3.26%	1.92%	94.81%

Figure 25: Daily regime probabilities

Figure 26: statistics.py

```
import pandas as pd
import yfinance as yf
import numpy as np
from scipy.stats import skew, kurtosis
import statsmodels.api as sm
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf
from statsmodels.tsa.stattools import adfuller

class Statistics:

    """
    Nourrir une section sur les propriétés statistiques des cryptos-actifs i.e. indicateurs de base
    sur les rendements des principaux tokens (BTC, ETH, XRP, ADA) et sortir des graphs.
    """

    def __init__(self, asset, time):
        self.asset = asset
        self.time = time
        self._prices = None
        self._returns = None
        self._logreturns = None
        self._variance = None
        self._volatility = None
        self._skew = None
        self._kurt = None

    def prices(self):
        self._prices = yf.download(self.asset, period=self.time).ffill().dropna()["Close"]

    def returns(self):
        self._returns = (self._prices - self._prices.shift(1)) / self._prices # self._prices.pct_change()

    def logreturns(self):
        self._logreturns = np.log(self._prices / self._prices.shift(1)) # np.log(self._prices.pct_change().tail(-
1))
```

```

def variance(self):
    self._variance = np.var(self._logreturns)

def volatility(self):
    self._volatility = round(self._logreturns.std() * 252 ** 0.5, 2)
    # df = pd.concat([self._variance, self._volatility], axis=1)
    # df.columns = ["Variance", "Volatility"]
    # print(df)

def mean_returns(self):
    """
    Moyenne des rendements sur plusieurs périodes (up, flat, down) pour justifier l'approche par
    régimes (moyenne des rendements en période de bull, bear, et flat)
    """

    print(self.asset)
    up = ("Up", "2020-09-27", "2021-03-08")
    flat = ("Flat", "2020-01-05", "2020-07-05")
    down = ("Down", "2021-11-07", "2022-06-12")

    # Building the dataframe
    log = list()
    vol = list()
    i = list()
    periods = [up, flat, down]
    for period in periods:
        prices = yf.download(self.asset, start=period[1], end=period[2]).ffill().dropna()["Close"]
        logret = np.log((prices / (prices.shift(1))))
        volat = logret.std() * 252 ** 0.5

        log.append(str(round(np.mean(logret) * 100, 2)) + '%')
        vol.append(str(round(np.mean(volat) * 100) + '%'))

        i.append(period[0])

    data = {'avg log': log, 'avg vol': vol, 'ETH-USD': i}
    print(pd.DataFrame.from_dict(data).set_index('ETH-USD').T)

```

```

def skewness_kurtosis(self):
    """
    https://www.geeksforgeeks.org/how-to-calculate-skewness-and-kurtosis-in-python/#
    - coefficient d'asymétrie (skewness)
    - coefficient d'aplatissement (kurtosis)
    """
    self._skew = skew(self._logreturns.tail(-1))
    self._kurt = kurtosis(self._logreturns.tail(-1))

    print(self._skew, self._kurt)

    data = {'Skewness': self._skew, 'Kurtosis': self._kurt}
    print(pd.DataFrame.from_dict(data).set_index(self._logreturns.columns))

def autocorr_ret(self):
    """
    - autocorrélation des rendements: corrélations entre  $r_t$  et  $r_{t-1}$ ,  $r_t$  et  $r_{t-2}$ , ...,  $r_t$  et  $r_{t-p}$ ,
    p=20.
    - print autocorrelation data
    - plot them and remove the first observation (i.e. 1) by zooming in
    """

    # autocorrelation data
    acorr = sm.tsa.acf(self._logreturns.tail(-1), nlags=30)
    print(acorr)

    # plot
    plt.plot(acorr)
    plot_acf(self._logreturns.tail(-1), lags=30) # lags=20
    plt.show()

def autocorr_abs(self):
    """
    - autocorrélation des valeurs absolues des rendements:
    corrélations entre  $|r_t|$  et  $|r_{t-1}|$ ,  $|r_t|$  et  $|r_{t-2}|$ , ...,  $|r_t|$  et  $|r_{t-p}|$ , p=20.
    Normalement, il doivent décroître vers zéro, mais lentement.
    Cela prouve que les rendements ne suivent pas un processus de marche aléatoire.
    """

```

```

# autocorrelation data
acorr = sm.tsa.acf(abs(self._logreturns.tail(-1)), nlags=30)
print(acorr)
plt.plot(acorr)

# plot
plot_acf(abs(self._logreturns.tail(-1)), lags=30) # lags=20
plt.show()

```

```

class PlottingData(Statistics):

```

```

    def __init__(self, asset, data, time):
        super().__init__(asset, time)
        self.prices()
        self.returns()
        self.logreturns()
        self.variance()
        self.volatility()
        # self.skewness_kurtosis()

        # print(self._prices)

        if data == "prices":
            # self.data = (self._prices / self._prices.iloc[0]) * 100
            self.data = self._prices

        elif data == "returns":
            self.data = self._returns.tail(-1)

        else:
            self.data = self._logreturns.tail(-1)

    def stationarity_tests(self):
        """
        Stationarity tests determine how strongly a time series is defined by a trend.
        Stationarity of returns will be rejected as p-values > 0.05
        Stationarity of logreturns will be accepted
        """

```

```

# easy
# result = adfuller(self.data)
# print('ADF Stastistic: %f' % stat)
# print('p-value: %f' % p)

# Building the dataframe
stats = list()
p = list()
for d in self.data:
    print(d)
    result = adfuller(self.data[d])
    stats.append(result[0])
    p.append(result[1])

data = {'ADF': stats, 'p-value': p}
print(pd.DataFrame.from_dict(data).set_index(self.data.columns))

def plotter(self):
    plt.plot(self.data, label=self.asset, linewidth=1, alpha=0.6)
    plt.legend(loc='upper left', fontsize=12)
    plt.title('price comparison')
    plt.show()

if __name__ == '__main__':
    cryptos = ["BTC-USD", "ETH-USD", "XRP-USD", "ADA-USD"]

    # run crypto by crypto for autocorrelations and mean_returns
    for crypto in cryptos:
        print(crypto)
        PlottingData(asset=crypto, data="returns", time="5Y").mean_returns()

    # PlottingData(asset=cryptos, data="logreturns", time="5Y").plotter()

```

Figure 27: model_5.py

```
from dataclasses import dataclass

import numpy as np
import pandas as pd
import yfinance as yf
from scipy.optimize import minimize
from scipy.stats import norm
import matplotlib.pyplot as plt

@dataclass
class MSGarchResult:
    mu: np.ndarray = None
    omega: np.ndarray = None
    alpha: np.ndarray = None
    beta: np.ndarray = None
    gamma: np.ndarray = None
    P: np.ndarray = None

def norm_pdf(x, mean=0, std_dev=1):
    return (1 / (np.sqrt(2 * np.pi * std_dev ** 2))) * np.exp(-(x - mean) ** 2 / (2 * std_dev ** 2))

def plot_probs(
    probs: np.ndarray
):
    row, regime = probs.shape
    fig, axes = plt.subplots(regime, figsize=(10, 7))

    for i in range(regime):
        ax = axes[i]
        ax.plot(probs[:, i])
        ax.set(title=f"Probability of regime {i}")
    fig.tight_layout()
    plt.savefig("Prob_per_regime.png")
```



```

class Model:

    def __init__(self, endog: np.ndarray):

        self.endog = endog
        if isinstance(endog, pd.DataFrame):
            self.data = endog.values
        if isinstance(endog, (list, tuple)):
            self.endog = np.asarray(endog)

        if self.endog is None:
            raise RuntimeError("Cannot fit a model without endog parameter")

        self.num_obs = len(endog)

    def fit(self):
        pass


class LikelihoodModel(Model):

    def fit(self, *args, **kwargs):
        """
        Fit a model to data.
        """
        if not hasattr(self, "log_likelihood"):
            raise RuntimeError("Cannot fit a likelihood model without self.log_likelihood function implemented")


class MSGarch(LikelihoodModel):

    def __init__(self, endog: np.ndarray, k_regimes: int = None, **kwargs):
        if k_regimes is None:
            self.k_regimes = 2
        else:
            self.k_regimes = k_regimes
        self.params = None

```

```

self.probs = None
self.sigma_2_k = None
self.sigma_2_t = None
self.loglik = None
self.mu = None
self.omega = None
self.beta = None
self.gamma = None
self.P = None

super().__init__(endog=endog)

def _prepare(self):
    pass

def log_likelihood(self, params, save: bool = False):
    T = len(self.endog)
    H = np.zeros((T, self.k_regimes))
    mu = params[:self.k_regimes]
    omega = params[self.k_regimes:2 * self.k_regimes]
    alpha = params[2 * self.k_regimes:3 * self.k_regimes]
    beta = params[3 * self.k_regimes:4 * self.k_regimes]
    tmp_gamma = params[4 * self.k_regimes:4 * self.k_regimes + (self.k_regimes * self.k_regimes)]

    gamma = np.zeros((self.k_regimes, self.k_regimes))
    # gamma = gamma.reshape(self.k_regimes, self.k_regimes) # doit etre vecteur colonne de taille 6 pour JD
    gamma[~np.eye(*gamma.shape, dtype=bool)] = tmp_gamma

    P = np.zeros((self.k_regimes, self.k_regimes))
    for i in range(0, self.k_regimes):
        for j in range(0, self.k_regimes):
            if i != j:
                P[i][j] = np.exp(gamma[i][j]) / (1 + np.sum(np.exp(gamma[i])) - np.exp(gamma.diagonal()[i]))
                # P[np.diag_indices_from(P)] = 1 / (1 + np.sum(np.exp(gamma[i])) - np.exp(gamma.diagonal()[i]))
    np.fill_diagonal(P, 1 - P.sum(axis=1))
    filtered_prob = np.zeros((T + 1, self.k_regimes))
    predict_prob = np.zeros((T, self.k_regimes))
    loglik = np.zeros((T, 1))

```

```

A = np.vstack((np.eye(self.k_regimes) - P, np.ones(self.k_regimes)))
I3 = np.eye(self.k_regimes + 1)
c3 = I3[:, -1]
# pinf = np.linalg.lstsq(A.T @ A, A.T @ c3, rcond=None)[0]
norm.pdf(np.random.normal(size=3))
pinf = np.ones(self.k_regimes) / self.k_regimes
filtered_prob[:2] = np.repeat(pinf[np.newaxis, :], 2, axis=0)
Ht = np.zeros(T)
H[0, :] = self.endog.var()
Ht[0] = pinf @ H[0, :]
for t in range(1, T):
    H[t, :] = omega + alpha * (self.endog[t - 1] - mu) ** 2 + H[t - 1, :] * beta

    LL = filtered_prob[t, :self.k_regimes] * norm.pdf(self.endog[t] - mu, 0, H[t, :self.k_regimes] ** (1 /
2))

    predict_prob[t, :] = LL / np.sum(LL)
    filtered_prob[t + 1, :] = P @ predict_prob[t, :]
    loglik[t, 0] = np.log(LL @ np.ones(self.k_regimes))
    Ht[t] = predict_prob[t, :] @ H[t, :]
print('alpha')
print(alpha)
print("beta")
print(beta)
print(sum(loglik))

if save:
    self.params = params

    self.probs = predict_prob
    self.sigma_2_k = H
    self.sigma_2_t = Ht
    self.loglik = loglik

    params = vars(MSGarchResult(mu, omega, alpha, beta, gamma, P))
    pd.DataFrame.from_dict(params, orient='index').to_csv(
        'params.csv')
    pd.DataFrame(self.probs).to_csv("probabilities.csv")
    pd.DataFrame(self.sigma_2_k).to_csv("sigma_2_per_regime.csv")
    pd.DataFrame(self.sigma_2_t).to_csv("sigma_2.csv")

```

```

        pd.DataFrame(self.loglik).to_csv("sigma_2.csv")
        plot_probs(self.probs)

    return -sum(loglik)

def _extract_params(self, params):

    pass

def fit(self):
    scaling_omega = 0 # different de 0!! # scaling omega = gamma sur note JD
    scaling_alpha = 0.15
    scaling_beta = 0.75 # WARNING: la somme de scaling_alpha et scaling_beta <1

    # Réfléchir à cette boucle
    compo = list()
    for i in range(0, self.k_regimes):
        compo.append(np.exp(((self.k_regimes / 2) - i)))

    vec = np.array(compo) # note Jean David page 3. C'est un vecteur de taille k de nombre de regime de marché
    omega_init = np.var(self.endog) * (1 - scaling_alpha - scaling_beta) * vec

    alpha_init = scaling_alpha * np.ones(self.k_regimes)
    beta_init = scaling_beta * np.ones(self.k_regimes)

    mu_init = np.ones(self.k_regimes) * np.mean(self.endog)
    mu_init[0] = -0.001
    mu_init[1] = 0
    mu_init[2] = 0.001

    # prob_reshape = matrix_prob.reshape((self.k_regimes * self.k_regimes))

    matrix_init = -3.5 * np.ones((self.k_regimes, self.k_regimes))

    prob_reshape = matrix_init[~np.eye(*matrix_init.shape, dtype=bool)]
    params_init = np.concatenate(
        (mu_init, omega_init, alpha_init, beta_init, prob_reshape)) # rajouter les mu

    bounds = []

```

```

bounds += [(-0.1, 0.1) for _ in range(len(mu_init))]
bounds += [(0, 1) for _ in range(len(omega_init))]
bounds += [(0, 0.25) for _ in range(len(alpha_init))]
bounds += [(0.7, 0.85) for _ in range(len(beta_init))]
bounds += [(-np.inf, np.inf) for _ in range(len(prob_reshape))]

bounds = tuple(bounds)  # tuple((-10, 10) for _ in range(params_init.shape[0]))

mu1_mu2_constraint = {'type': 'ineq',
                      'fun': lambda params: params[1] - params[0]}

mu2_mu3_constraint = {'type': 'ineq',
                      'fun': lambda params: params[2] - params[1]}

ab1_constraint = {'type': 'ineq',
                  'fun': lambda params: params[7] + params[10] - np.ones(self.k_regimes)}

ab2_constraint = {'type': 'ineq',
                  'fun': lambda params: params[8] + params[11] - np.ones(self.k_regimes) }

ab3_constraint = {'type': 'ineq',
                  'fun': lambda params: params[9] + params[12] - np.ones(self.k_regimes)}

# todo: alpha + beta <= 1

res = minimize(
    self.log_likelihood, params_init,
    method="Nelder-Mead",
    constraints=(mu1_mu2_constraint, mu2_mu3_constraint, ab1_constraint, ab2_constraint, ab3_constraint),
    bounds=bounds,
    tol=1e-1,
    options={"disp": True}
)
self.log_likelihood(res.x, save=True)
print(res.x)

pass

```

```
if __name__ == '__main__':  
    btc_usd = yf.Ticker("BTC-USD")  
    history = btc_usd.history(period="10Y")  
    log_returns = np.diff(np.log(history["Close"]))  
    ms_garch = MSGarch(log_returns, 3)  
    ms_garch.fit()
```